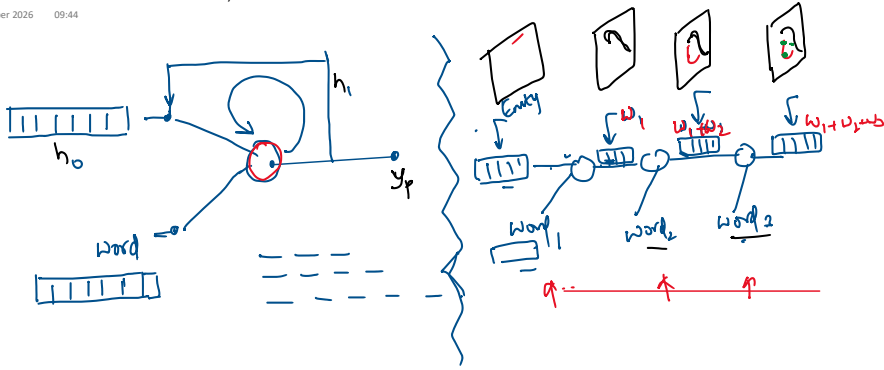


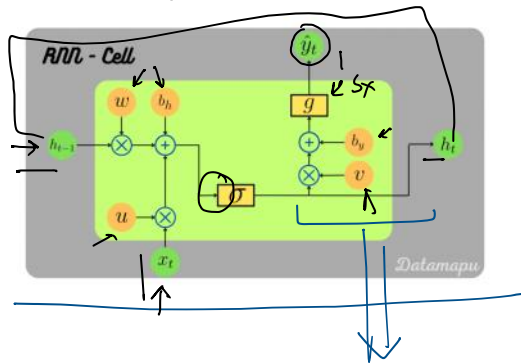


$$w_1 \text{word} + w_2 \text{Empty} + \text{bias} \rightarrow y_f$$

$$w_1 x_1 + w_2 x_2 + b \rightarrow y_f$$

$$u x_t + w h_{t-1} + b_h \rightarrow -ve \rightarrow 0$$

$$\sigma \begin{bmatrix} 0.1 \\ 2 \\ 3 \\ 9 \\ 8 \\ 7 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.3 \\ 0.5 \\ 0.8 \\ 0.9 \\ 0.7 \end{bmatrix}$$



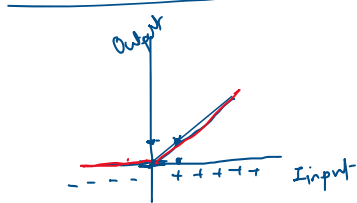
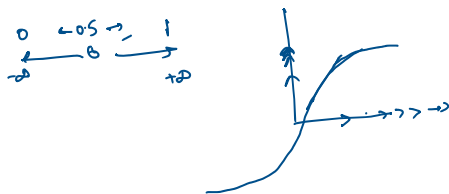
empty array
→ hidden-state

$$\phi \rightarrow \text{family} \rightarrow \begin{bmatrix} -1 \\ 1 \\ -2 \end{bmatrix} \quad \begin{bmatrix} +1 \\ 1 \\ +2 \end{bmatrix}$$

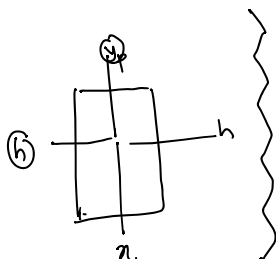
$$\begin{matrix} -1000 & +100 \\ \downarrow & \downarrow \\ -0.99 & \sim 0.97 \end{matrix}$$

$$w_{11} h_t + b_y \rightarrow \text{ReLU} \rightarrow y_t$$

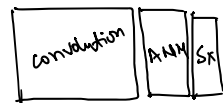
$$A \rightarrow \begin{matrix} \rightarrow \sigma \\ \rightarrow S_{\max} \\ \rightarrow R \end{matrix} \left. \begin{matrix} \text{probabilities} \\ \text{[-ve} \rightarrow 0 \end{matrix} \right\}$$

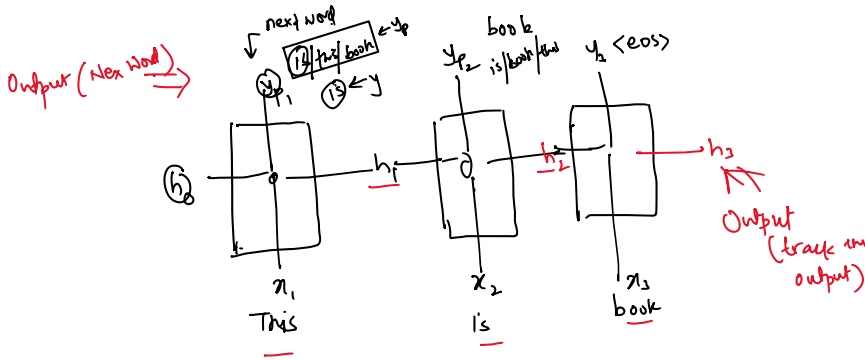
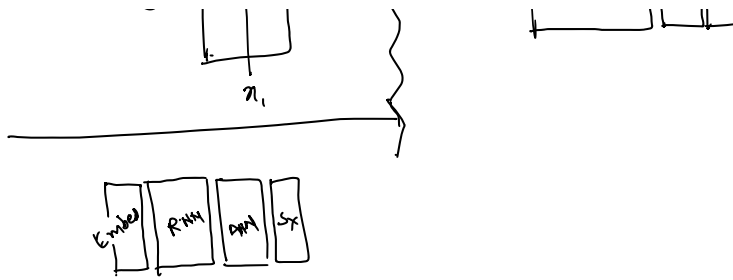


RNN



CNN



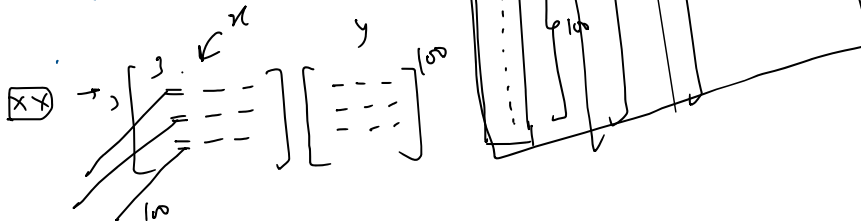


Vocab $\rightarrow$ Unique words



```
class NextWordRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim=100, hidden_dim=16):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x):
        x = self.embedding(x) # [3, 3, 100]
        out, hx = self.rnn(x) # [3, 16], [1, 3, 16]
        out = out[:, :, :] # [3, 16]
        out = self.fc(out) # [3, 10]
        return out
```



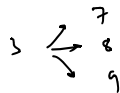
```
word2idx = {w:i for i,w in enumerate(vocab)}
idx2word = {i:w for w,i in word2idx.items()}
idx2word = {i:w for i,w in enumerate(vocab)}

idx2word
{0: 'algorithm',
 1: 'fun',
 2: 'great',
 3: 'i',
 4: 'implementing',
 5: 'is',
 6: 'learning',
 7: 'love',
 8: 'machine',
 9: 'word2vec'}
```

for w in (vocab)
- print(w)

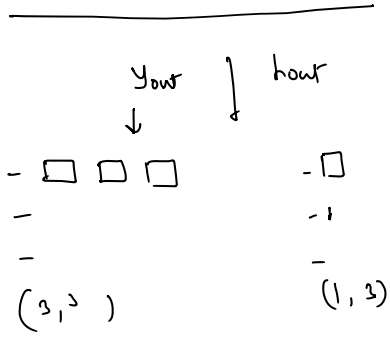
```
x = tensor([3, 7, 8, 9, 5, 2, 4, 9, 5])
y = tensor([7, 8, 6, 5, 2, 9, 5, 1])
```

```
corpus = [
    "I love machine learning",
    "word2vec is great algorithm",
    "Implementing word2vec is fun"
]
```



I love machine
love machine learn

$x_{\text{word}}, h_1 = \text{self.rnn}(x, x_0)$



```

self.rnn = nn.RNN(embed_dim, hidden_dim, batch_first=True)
self.fc = nn.Linear(hidden_dim, vocab_size)

```

Diagram showing the flow of data from input x through the RNN and fully connected layer to produce output out .

Code snippet:

```

x = self.embedding(x) # [3, 4, 100]
out, hx = self.rnn(x, hx) # [3, 4, 16], [1, 3, 16]
out = out.reshape(-1, out.shape[2]) # [12, 16] # for next word
out = self.fc(out) # [12, 13]
return out

```

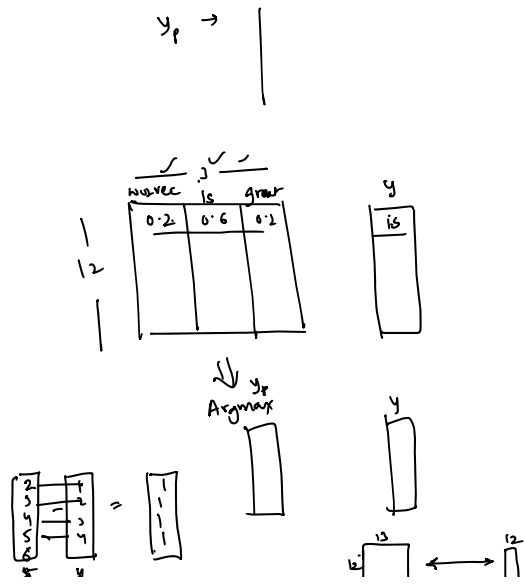
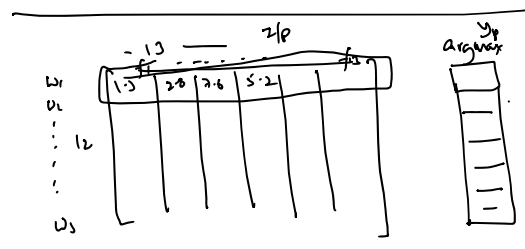
Handwritten notes:

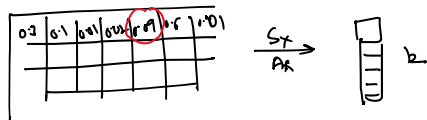
- $as = (2, 3, 4) = 24$
- $\downarrow \text{reshape}$
- $as.reshape(2, -1)$
- $(2, 12)$
- $\circ \text{reshape}(15, -1)$
- $(15, 8)$
- $(5, 24)$

Handwritten equations:

$$out = nn.linear() \cdot [x, 12]$$

$$out = nn.linear() \cdot [x, 10]$$



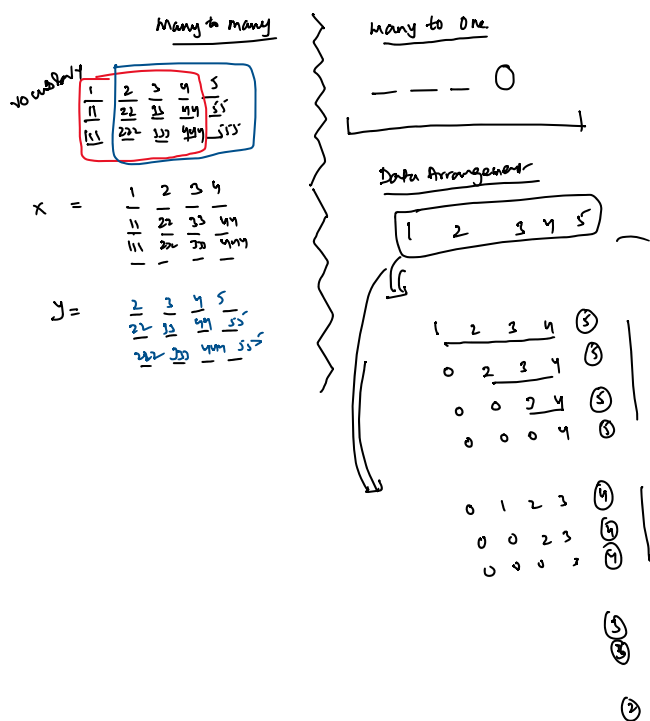
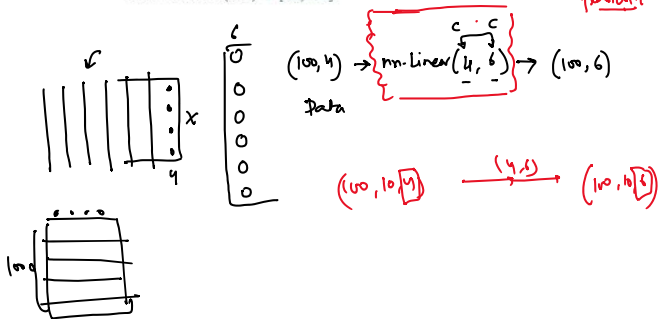


```
def forward(self, x, hx=None):
    x = self.embedding(x) # [3, 4, 100]
    out, hx = self.rnn(x, hx) # [3, 4] [6], [1, 3, 16]
    out = self.reshape(-1, out.shape[-1]) # [12, 16] # for next
    out = self.fc(out) # [12, 13]
    return out
```

`_first=True)`

`out = shape` → (3, 4, 16)

→ Masse flattent!



- Special Data Arrangement
 - Padding
 - Training $\times$
- $\Rightarrow$ Many 2 One

 $\chi = []$
$$X = []$$
$$\text{range}(1, 5) \rightarrow 4$$

```

X = []
Y = []
for sent in sentences:
    for i in range(len(sent)-1):
        X.append(word2idx[sent[i]])
        Y.append(word2idx[sent[i+1]])

```

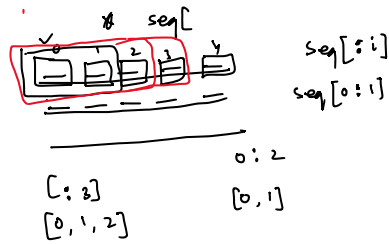
```

X = []
Y = []
for seq in sequences:
    for i in range(1, len(seq)):
        X.append(seq[i-1])
        Y.append(seq[i])

```

range(1, 5) → 4
i=1

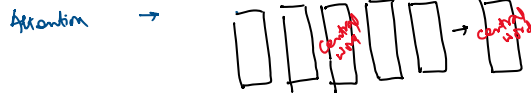
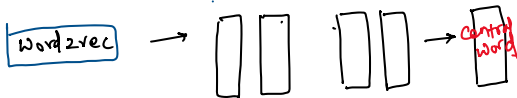
range(s) → 5
(2, 5) → 2



[0] [1]
[0, 1] [2]
[0, 1, 2] [3]
[0, 1, 2, 3] [4]

0 0 0 0
0 0 0 0
0 1 2 3
0 2 3 3

Attention

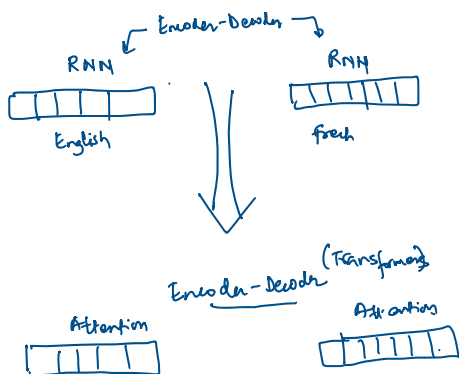


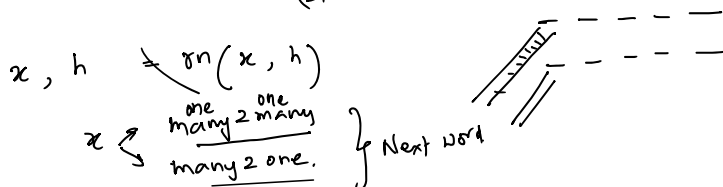
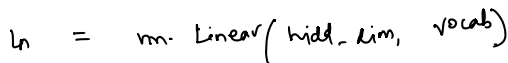
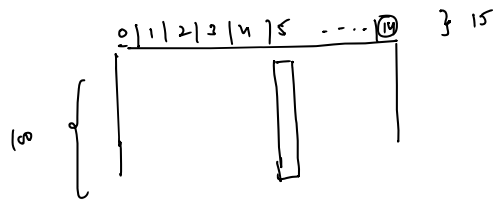
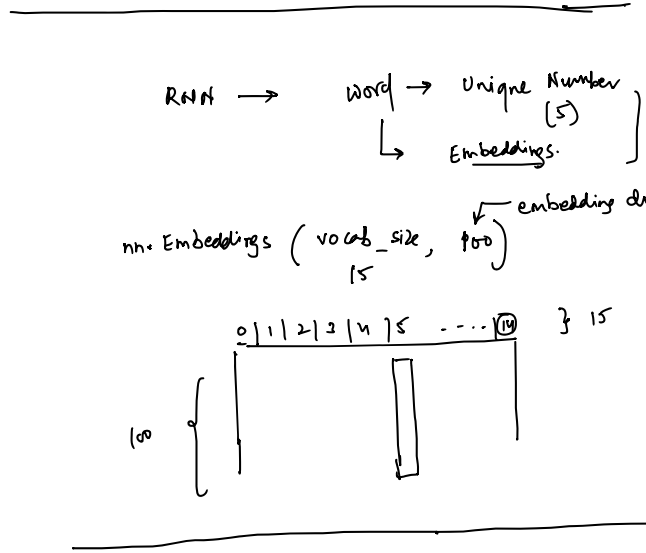
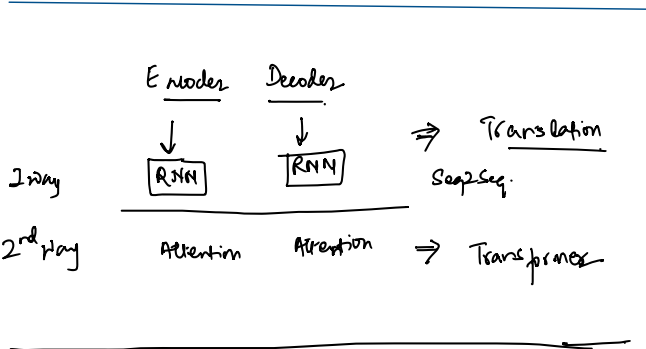
RNN →

word₁ → word₂ → words
All the words of sentence

Attention →

All the words are ingested
in Linear Combination
thus one shot

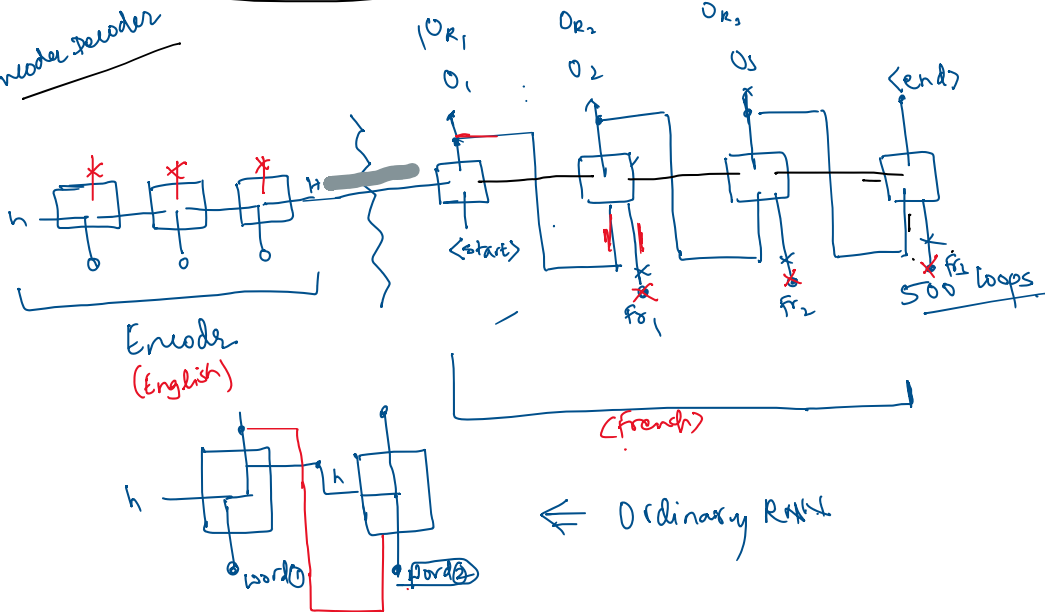




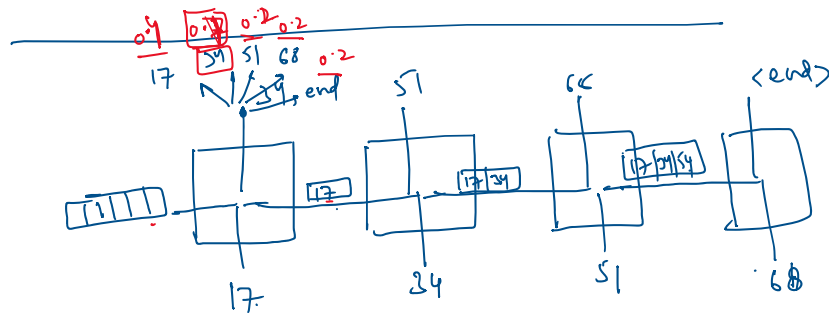
New Section 1 Page 6

$\pi, [h] = \text{rnn}(\pi, h)$
 $\pi \rightarrow \text{reshape}(-1, \pi.\text{shape}[2]) \rightarrow \begin{matrix} \text{(one zone)} \\ \text{many 2 many} \end{matrix}$
 $\pi \rightarrow \pi[:, :, :] \rightarrow \text{many zone.}$

Encoder Decoder



Ratio $\rightarrow$
 one zone [loop]



Likelihood

y	y _p	y _p	loss
●	P(●)	0.7	0.3
●	P(●)	0.8	0.2

y	y _p
●	P(●) = 0.7
●	P(●) = 0.2
●	P(●) = 0.8

Likelihood loss error

$$\text{Likelihood loss} = \frac{\text{Sum Error}}{n}$$

Log Likelihood Loss

$$\underbrace{-\log(0.3)}_{=1.6} \rightarrow 3.8$$

— Optimization → Loss

Negative Log Likelihood Loss

NLL Loss + $\log$ Softmax

Cross Entropy

`np.random(1)`

`teacher_force = torch.rand(1).item() < 0.5 teacher_forcing_ratio`

if `np.random(1)` < ~~0.5~~ _{0.8}

output prob
↓
Input

`np.random.uniform()`

`np.random.normal(1)`

[... 0 ...]

↑↑↑↑↑↑↑↑↑↑

→ ∞

```
def forward(self, target_seq, hidden, teacher_forcing_ratio=0.5):
    batch_size, target_len = target_seq.size()
    outputs = []

    input_token = target_seq[:, 0].unsqueeze(1) # first input (<SOS> assumed)

    for t in range(1, target_len):
        embedded = self.embedding(input_token) # (batch, 1, embed_dim)
        output, hidden = self.rnn(embedded, hidden) # (batch, 1, hidden)
        logits = self.fc(output.squeeze(1)) # (batch, vocab)
        outputs.append(logits.unsqueeze(1)) # collect for all time steps

    teacher_force = torch.rand(1).item() < teacher_forcing_ratio
    next_input = target_seq[:, t] if teacher_force else logits.argmax(1)
    input_token = next_input.unsqueeze(1)

    return torch.cat(outputs, dim=1) # (batch, target_len-1, vocab)
```

[] target [0:t] → [t]

if `np.random.rand(1) > 0.7`:

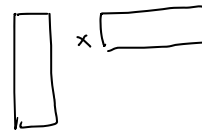
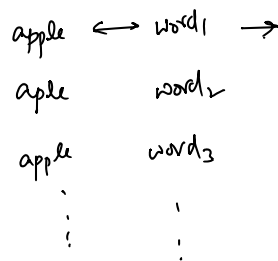
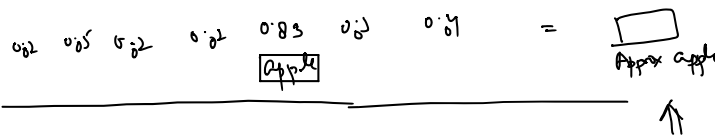
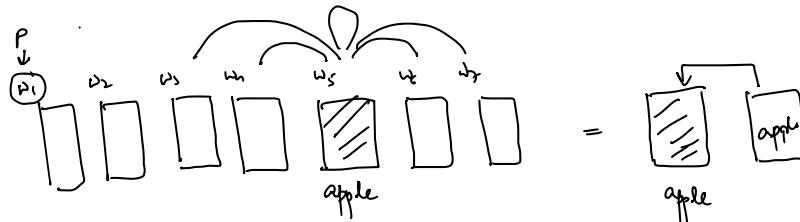
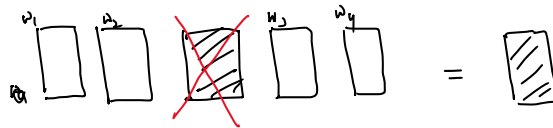
word by word → as input

else

previous_output → as input

Attention

Attention



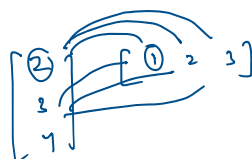
$$[2, 3, 4] \rightarrow (1, 3)$$

$$[2, 3, 4] [5, 4, 7] \rightarrow (2, 2)$$

$$[2, 3, 4] \xrightarrow{\text{perm}} [2] [3] [4] \quad (3, 1)$$

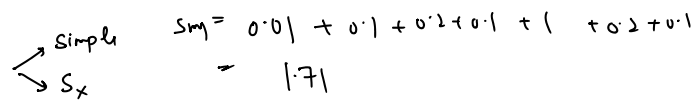
$$[x, x] [x, x] [x, x] \quad (3, 2)$$

$$[2], [3], [4] \rightarrow \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} \quad (3, 1) \quad \uparrow \text{row, 1 col}$$



$$(1, 3) \times (3, 1) \Rightarrow [1, 1] \leftarrow$$

$$(3, 1) \times (1, 3) \Rightarrow [3, 3]$$



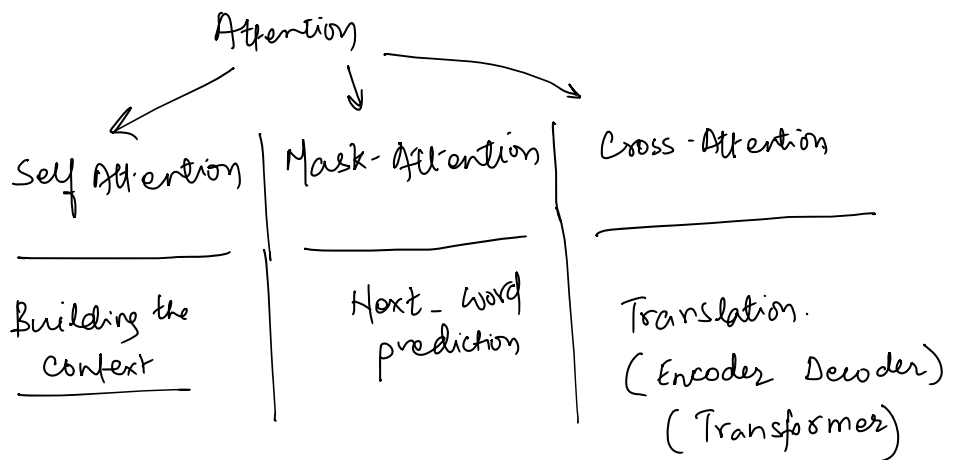
$$[10, 11] = \frac{e^{10}}{e^{10} + e^{11}}, \quad \frac{e^{11}}{e^{10} + e^{11}}$$

$$= 0.26 + 0.74 = 1$$



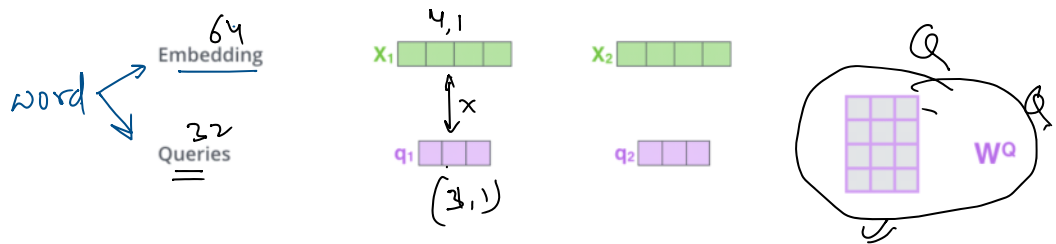
New Section 1 Page 10

value.



This is an apple

① ② ③ ④



embed word × Key em → W_K

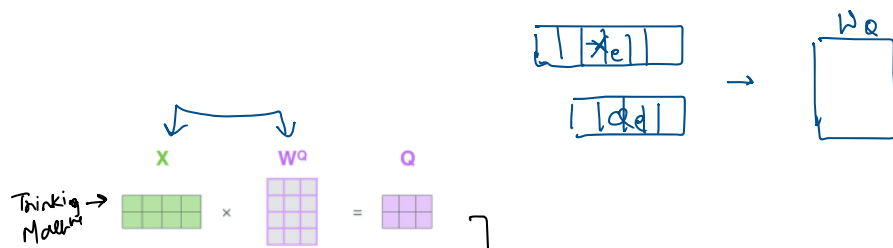
Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$

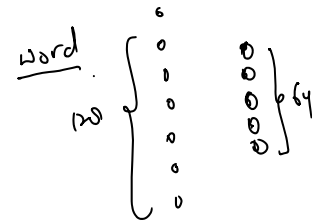
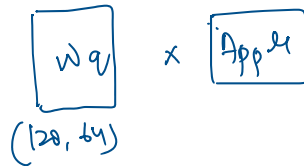
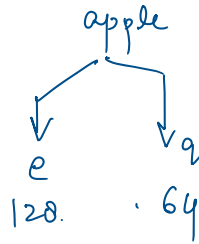
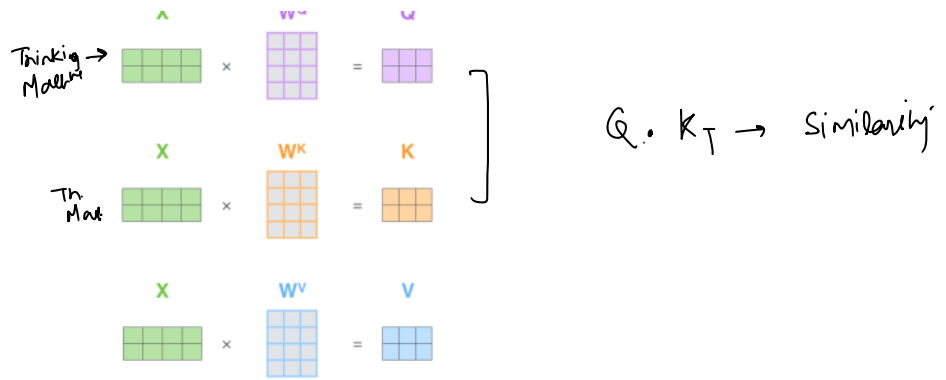
Thinking

$e \times q = Q$

$e \times k = K$

$e \times v = V$





```

Q self.query_linear = nn.Linear(128, 64)
K self.value_linear = nn.Linear(128, 64)
K self.key_linear = nn.Linear(128, 64)
self.out_linear = nn.Linear(64, 64)

```

```

def forward(self, x):
    # (32, 10, 128)
    query = self.query_linear(x) # (32, 10, 64)
    key = self.key_linear(x)
    value = self.value_linear(x)

    # print(query.shape, key.transpose(-2, -1).shape)

    x = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(64)
    # print(x.shape)
    x = torch.softmax(x, dim=-1)
    x = torch.matmul(x, value)
    x = self.out_linear(x)
    return x

```

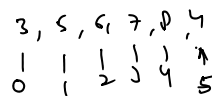
Compensated Similarity

Similarity

```

: torch.zeros(3, 4, 5, 6, 7, 8).transpose(1, 0).shape
: torch.Size([3, 7, 5, 6, 4, 8])

```



Next word prediction

Mask Attention

